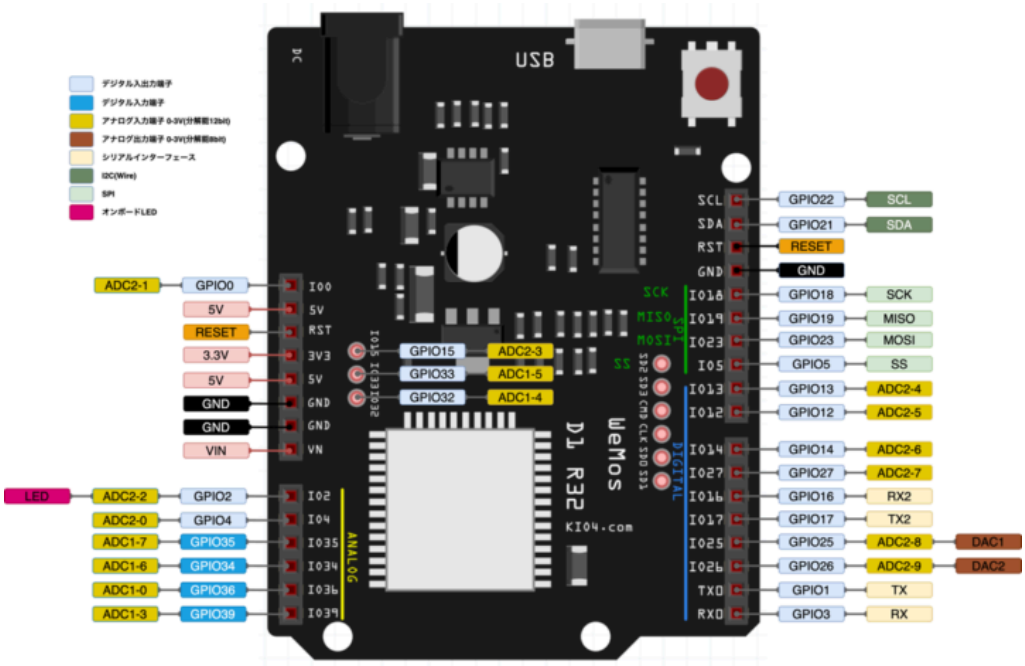
Płytka WEMOS ESP32 R32 (D1 UNO R32) to moduł deweloperski oparty na układzie ESP32, zachowujący częściową zgodność z wyprowadzeń z Arduino Leonardo. Umożliwia korzystanie z popularnych shieldów, jednocześnie oferując znacznie większą wydajność i wbudowaną łączność bezprzewodową. Sprawdza się w projektach IoT, automatyce oraz systemach inteligentnego domu. Dzięki dwurdzeniowemu procesorowi Xtensa LX6 może wykonywać wiele zadań równolegle, a Wi-Fi i Bluetooth eliminują potrzebę dodatkowych modułów komunikacyjnych.



Parametry techniczne:

- Mikrokontroler: ESP32 (dual-core, 32-bit)
- Taktowanie: 80–240 MHz
- RAM: 520 kB
- Flash: 4 MB
- Wi-Fi: 802.11 b/g/n
- Bluetooth: 4.2 + BLE

Wejścia/wyjścia:

- GPIO: 30 pinów (nie wszystkie wyprowadzone na złącza UNO)
- ADC: 18 kanałów, rozdzielczość 12-bit
- DAC: 2 kanały, rozdzielczość 8-bit
- Interfejsy: UART, SPI, I2C, PWM

Zasilanie:

- Napięcie zasilania: 5–12 V (VIN)
- Logika: 3.3 V

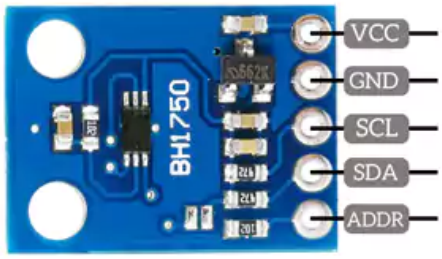
Zad 1. W środkowisku Arduino IDE należy dodać nastepujące biblioteki:

- `ESPAsyncTCP`
- `ESPAsyncWebServer`

Dodaj, o ile to konieczne, obsługę mikrokontrolera esp32 do środowiska programistycznego - Arduino IDE.

Zad 2. Impelementacja punktu dostępowego (AP). Kod znajduje się w pliku `eth02_esp32_ap.ino` (plik `eth02_esp32.zip`). Do płytki ESP32 R32 (ESP32 Dev Module w środowisku Arduino IDE) należy podłączyć przycisk (GPIO 2) oraz LED (GPIO 11). Zaprogramować układ ESP32 i sprawdzić działanie programu.

Zad 3. Zbuduj prototyp miernika natężenia oświetlenia, korzystając z modułu BH1750.



Zasilanie **+3.3V**. Linia `ADDR` może pozstać niepodłączona. Kod znajduje się w pliku `eth02_esp32_webpage_bh1750.ino` (plik `eth02_esp32.zip`).

```
#include<ESP8266WiFi.h>
#include<ESPAsyncTCP.h>
#include<ESPAsyncWebServer.h>
#include<Wire.h>
#include<BH1750.h>
#define BAUDRATE 115200

const char* ssid = "WSG-Mobilna pracownia";
const char* pass = "M0b1lnaPracown1a";

AsyncWebServer server(80);

BH1750 light_sensor;
double lux = 0;

const char index_html[] PROGMEM = R"webpage(
<!DOCTYPE html><html>
<head>
  <meta name='viewport' content='width=device-width,
initial-scale=1.0'>
  <title>ESP8266 Lux meter</title>
  <link
href='https://fonts.googleapis.com/css?family=Open+Sans:300,400
,600' rel='stylesheet'>
  <style>
   html { font-family: 'Open Sans', sans-serif; display: block;
margin: 0px auto; text-align: center;color: #444444;}"
   body{margin: 0px;}"
   h1 {margin: 50px auto 30px;}
   .side-by-side{display: table-cell;vertical-align:
middle;position: relative;}
   .text{font-weight: 600;font-size: 19px;width: 200px;}
   .reading{font-weight: 200;font-size: 50px;padding-right:
25px;}
   .light.reading{color: #F29C1F;}
   .superscript{font-size: 17px;font-weight: 600;position:
absolute;top: 10px;}
   .data{padding: 10px;}
   .container{display: table;margin: 0 auto;}
```

```
   .icon{width:65px}
   </style>
 <script>
  setInterval(updateLux,1000);
  function updateLux() {
    var xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
      if(this.readyState == 4 && this.status == 200) {
        document.getElementById("light_int").innerHTML =
        this.responseText;} };
    xhttp.open("GET","light", true);
    xhttp.send(); }
 </script>
</head>
<body>
  <h1>ESP8266 Lux meter</h1>
  <div class='container'>
  <div class='data light'>
  <div class='side-by-side icon'>
  <svg xmlns="http://www.w3.org/2000/svg"
xmlns:xlink="http://www.w3.org/1999/xlink" version="1.1"
width="60" height="60" viewBox="0 0 60 60" xml:space="preserve">
  <g transform="matrix(0.39 0 0 0.39 29 29)"><path style="stroke:
rgb(238,238,40);
  stroke-width: 8; stroke-dasharray: none; stroke-linecap: butt;
stroke-dashoffset: 0;
  stroke-linejoin: miter; stroke-miterlimit: 10; fill:
rgb(238,238,40); fill-rule: evenodd; opacity: 1;"
  transform=" translate(-59.83, -59.83)" d="m 0 59.82677 l
27.527777 -8.654488 l -19.512508 -21.258898 l 28.167 6.268881 l
-6.268881 -28.167 l 21.258898 19.512508 l 8.654488 -27.527777 l
8.654491 27.527777 l 21.258896 -19.512508 l -6.2688828 28.167 l
28.167 -6.268881 l -19.512512 21.258898 l 27.527779 8.654488 l
-27.527779 8.654491 l 19.512512 21.258896 l -28.167 -6.2688828 l
6.2688828 28.167 l -21.258896 -19.512512 l
  -8.654491 27.527779 l -8.654488 -27.527779 l -21.258898
19.512512 l 6.268881 -28.167 l -28.167 6.2688828 l 19.512508
-21.258896 z" stroke-linecap="round" />
  </g></svg>
  </div>
  <div class='side-by-side text'>Light intensity</div>
  <div class='side-by-side reading'>
  <span id = "light_int"> 0 </span>
```

```
  <span class='superscript'>lx</span></div>
  </div>
</body></html>
)webpage";

void setup() {
Serial.begin(BAUDRATE);
WiFi.begin(ssid,pass);
Serial.print("Connecting to "); Serial.print(ssid);
Serial.print("with password "); Serial.println(pass);
Serial.print("Start connection .");
while(WiFi.status()!=WL_CONNECTED) {
  delay(500);
  Serial.print("."); }
Serial.print("ESP8266 board IP ");
Serial.print(WiFi.localIP());

Wire.begin();
light_sensor.begin();

//web page
server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
    request->send_P(200, "text/html", index_html);
  });

server.on("/light", HTTP_GET, [](AsyncWebServerRequest
*request){
  request->send(200, "text/plain", String(lux));
  });

server.begin();
}

long p_millis = 0;
int i=0;
String message;
#define HR_MODE_DELAY 1000 // High Resolution Mode min.120[ms]

void loop() {
if(millis() - p_millis > HR_MODE_DELAY) {
  lux = light_sensor.readLightLevel();
  message = "Light intensity" + String(lux);
  Serial.println(message + "[lx]");
```

```
      p_millis = millis();
      }
    }
```

Uwaga. Fragmentu kodu wyróżniego mniejszą czcionką i kursywą nie należy przepisywać. Wspomniany kod jest grafiką w formacie SVG i rozmiarach 50px x 50px.

Została ona wygenrowana przy pomocy ***svgeditoronline.com/editor/***. Do celów realizacji zadania należy przygotować swoją grafikę.

Zad 5. Zapis danych do pamięci nieulotnej. Kod znajduje się w pliku `eth02_esp32_webpage_bh1750.ino` (plik `eth02_esp32.zip`).

```
#include <ArduinoJson.h> // Library for JSON serialization and deserialization
#include <EEPROM.h>      // Library for flash memory access

// Define the EEPROM size (max 512 bytes for ESP32)
#define EEPROM_SIZE 512

void setup() {
  Serial.begin(115200);

  // Initialize EEPROM
  if (!EEPROM.begin(EEPROM_SIZE)) {
    Serial.println("Failed to initialize EEPROM");
    return;
  }

  Serial.println("Reading data from flash memory...");

  String storedJson = "";
```

```
  for (int i = 0; i < EEPROM_SIZE; i++) {
    char character = char(EEPROM.read(i));
    if (character == '\0') break;
    storedJson += character;
  }

  if (storedJson.length() > 0) {
    JsonDocument doc;
    DeserializationError error = deserializeJson(doc, storedJson);

    if (!error) {
      const char* user_id = doc["user_id"];
      const char* password = doc["password"];

      Serial.print("Data retrieved from flash: ");
      Serial.println(storedJson);
      Serial.print("User ID: "); Serial.println(user_id);
      Serial.print("Password: "); Serial.println(password);
    } else {
      Serial.print("JSON Deserialization failed: ");
      Serial.println(error.c_str());
    }
  } else {
    Serial.println("No data found in flash memory.");
  }
}

void loop() {
  JsonDocument doc;
  doc["user_id"] = "admin248";
  doc["password"] = "pass248";

  String jsonString;
  serializeJson(doc, jsonString);

  Serial.print("Saving to flash: ");
  Serial.println(jsonString);
```

```
  for (int i = 0; i < jsonString.length(); i++) {
    EEPROM.write(i, jsonString[i]);
  }
  EEPROM.write(jsonString.length(), '\0');

  if (EEPROM.commit()) {
    Serial.println("Data successfully saved to flash!");
  } else {
    Serial.println("Flash save failed!");
  }

  while(1);
}
```

Zad 5. Przygotować prototyp urządzenia, które działa w następujący sposób:

- (0.1 pkt.) Po uruchomieniu urządzenie uruchamia się w trybie puntu dostępowego. Użytkownik może wprowadzić parametry niezbędne do połączenia z siecią bezprzewodową. Parametry muszą być zapisane w pamięci flash modułu.
- (0.2 pkt.) Po ustawieniu parametrów połączenia WiFi urządzenie powinno połączyć sie ze skonfigurowaną siecią bezprzewodową (można wykorzystać mechanizm reset). Połączenie z siecią bezprzewodową powinno następować po każdym resecie urządzenia.
- (0.2 pkt.) Dodać przycisk do kasowania ustawień połączenia.
- (0.5 pkt.) Urządzenie udostępnia stronę www na której jest wyświetlona wartość natężenia oświetlenia w formie numerycznej (0.2 pkt.) i graficznej (wykres lub wskaźnik analogowy) (0.2 pkt.). Strona działa w sposób asynchroniczny - automatyczne odświeżanie (0.1 pkt).